

Full Name: Abdul Hadi Khan
Student Number: 1007984736
UTORid: khanab72
Course: CSC343H5 · Introduction to Databases
Sprint: 5 - Transactions & Concurrency
Submission: First submission

1. Precedence Graphs and Serializability

1.1. Schedule S1

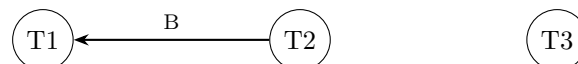
S1 = R1(A) R2(B) W1(A) R3(C) W2(B) R1(B) W3(C) W1(B)

(i) Conflicting pairs

Object	Pair, in schedule order	Type	Edge
B	R2(B), W1(B)	RW	T2 → T1
B	W2(B), R1(B)	WR	T2 → T1
B	W2(B), W1(B)	WW	T2 → T1

Note: A is touched only by T1 (R1(A), W1(A)), and C is touched only by T3 (R3(C), W3(C)), so neither object produces a conflict between two different transactions. T3 conflicts with nothing in this schedule.

(ii) Precedence graph



All three conflicting pairs on B (RW, WR, WW) point the same direction, so they collapse to one edge T2 → T1 in the precedence graph; T3 has no edges at all.

(iii) Verdict

Conflict serializable. All three edges point the same direction, T2 → T1, and T3 has no edges at all, so the graph is acyclic. The only constraint is that T2 must precede T1; T3 can be placed anywhere. This gives three equivalent serial orders: T3,T2,T1; T2,T3,T1; T2,T1,T3. The order is therefore **not unique** — T3's position is unconstrained because T3 conflicts with neither T1 nor T2.

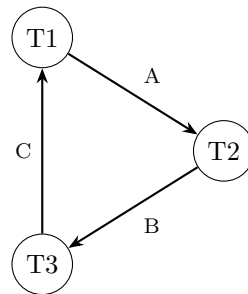
1.2. Schedule S2

S2 = R1(A) W2(A) R2(B) W3(B) R3(C) W1(C)

(i) Conflicting pairs

Object	Pair, in schedule order	Type	Edge
A	R1(A), W2(A)	RW	T1 → T2
B	R2(B), W3(B)	RW	T2 → T3
C	R3(C), W1(C)	RW	T3 → T1

(ii) Precedence graph



(iii) Verdict

Not conflict serializable. Every transaction conflicts with exactly one other transaction, and the edges form the cycle $T1 \rightarrow T2 \rightarrow T3 \rightarrow T1$. No serial order can exist because a serial order is a total order on $\{T1, T2, T3\}$, and any total order must place one of these three transactions first, last, and in the middle — but the cycle requires each transaction to precede another that must, in turn, precede it, which no linear order can satisfy.

2. Aborts and Recoverability

S3 = W1(A) R2(A) W2(B) C1 C2 S4 = W1(A) R2(A) W2(B) C2 C1

S5 = W1(A) W2(A) C1 C2 S6 = W1(A) R2(A) W2(B) R3(B) W3(C) A1

(i) Classification

Schedule	Recoverable?	ACA?	Strict?	Deciding pair
S3	Yes	No	No	W1(A) then R2(A)
S4	No	No	No	C2 then C1
S5	Yes	Yes	No	W1(A) then W2(A)

S3: T2 reads A from T1 before T1 commits (dirty read at W1(A) then R2(A)), so S3 is not ACA and not strict; it is recoverable because the reader, T2, commits (C2) only after the writer it depends on, T1, commits (C1).

S4: identical dirty read, but now T2 commits (C2) before T1 commits (C1) – the deciding pair. A transaction that depends on uncommitted data has committed before the transaction it depends on, so S4 is not recoverable (and hence not ACA, not strict).

S5: T2 blindly overwrites A with W2(A) while T1's write is still uncommitted, but T2 never reads A, so there is no reads-from dependency to satisfy: S5 is recoverable and ACA (vacuously, since it has no dirty read at all). It fails strictness, because Strict requires no transaction write to data written by an uncommitted transaction, and W2(A) does exactly that while W1(A) is uncommitted.

(ii) S5: which property fails, and why undo gets harder

*S5 fails **strictness**: T2's write W2(A) lands on top of T1's uncommitted write, even though T2 never reads A. This makes undo harder because if T1 aborts, the DBMS cannot simply restore A to its before-image (the value before W1(A)), since that would silently discard T2's write as well; a correct undo now has to reason about which later writes must also be compensated, instead of a simple physical rollback.*

(iii) The cascade in S6

S6 = W1(A) R2(A) W2(B) R3(B) W3(C) A1

1. **T1** – aborts directly (A1); this is the initiating abort.
2. **T2** – must be rolled back because R2(A) read the value A wrote by T1, and T1 subsequently aborted (a dirty read forces a cascading abort of the reader).
3. **T3** – must be rolled back because R3(B) read the value B wrote by T2, and T2 is now being rolled back in turn (the cascade propagates transitively).

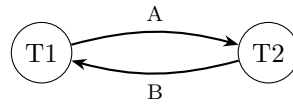
Rollback order: T1, then T2, then T3.

If **ACA (Avoiding Cascading Aborts)** had been enforced – i.e. no transaction may read data written by an uncommitted transaction – then T2 could never have read T1's uncommitted A, and T3 could never have read T2's uncommitted B, so this entire cascade could not have happened.

3. Locking and Two-Phase Locking

S7 = X1(A) R1(A) W1(A) U1(A) X2(A) R2(A) W2(A) X2(B) R2(B) W2(B)
 U2(A) U2(B) X1(B) R1(B) W1(B) U1(B) C1 C2

(i) The cycle



On A: $W1(A)$ occurs (position 3) before $R2(A)$ and $W2(A)$ (positions 6–7), giving WR and WW conflicts, both $T1 \rightarrow T2$. On B: $R2(B)$ and $W2(B)$ (positions 9–10) occur before $R1(B)$ and $W1(B)$ (positions 14–15), giving RW, WR, and WW conflicts, all $T2 \rightarrow T1$. The graph is the 2-cycle $T1 \rightarrow T2 \rightarrow T1$ – every lock is legally held one at a time, yet $T1$'s writes to A are seen by $T2$ while $T2$'s writes to B are seen by $T1$, so neither transaction's operations can be ordered before the other's.

(ii) 2PL and Strict 2PL verdicts

Transaction	2PL?	Strict 2PL?	Deciding operation, and why
T1	No	No	$X1(B)$: acquired at position 13, <i>after</i> T1 had already released a lock ($U1(A)$, position 4) – a lock acquired after a release violates the growing-phase rule of 2PL, so T1 fails 2PL outright (and therefore Strict 2PL too).
T2	Yes	No	$U2(A)$: released at position 11, <i>before</i> C2 (position 18). T2's two acquisitions ($X2(A)$, $X2(B)$) both precede its two releases ($U2(A)$, $U2(B)$), so it obeys ordinary 2PL; but Strict 2PL requires every lock to be held until commit, and $U2(A)$ releases early.

(iii) Strict 2PL rewrite

T1	T2
X1(A)	
R1(A)	
W1(A)	
	X2(A) – requested, DENIED and WAITS (T1 still holds the X-lock on A)
X1(B)	
R1(B)	
W1(B)	
C1	
U1(A)	
U1(B)	<i>(T2's pending X2(A) is now granted, since U1(A) just released it)</i>
	X2(A) – granted
	R2(A)
	W2(A)
	X2(B) – granted (B is free; T1 released U1(B) above)
	R2(B)
	W2(B)
	C2
	U2(A)
	U2(B)

Equivalent serial order: T1, T2. Every lock T1 holds is kept until C1, and every lock T2 holds is kept until C2, so both transactions are Strict 2PL. T2's request for A is denied while T1 holds it and only granted after T1's commit-time release, which is exactly why the resulting schedule is serializable in the order T1 then T2.

(iv) Deadlock under Strict 2PL

Suppose T1 acquires an X-lock on A and then requests an X-lock on B, while concurrently T2 has already acquired an X-lock on B and then requests an X-lock on A: each transaction holds the object the other needs and, because Strict 2PL forbids releasing a lock early to let the other proceed, both wait forever. The DBMS detects the cycle in its wait-for graph (or times a transaction out) and breaks the deadlock by choosing one transaction as the victim and aborting it, releasing its locks so the other can continue.

4. Choosing an Isolation Level

Per the instructor's email announcement, Questions 4 and 5 (Isolation Levels) have been removed from Sprint 5 grading: "No Isolation Levels (Question 4 and 5) on Sprint 5 Anymore. They were not covered in lecture in as much detail as we'd want for this question, so I'm not looking for responses to these questions!" No response is submitted for this section.

5. A Read–Decide–Write Hazard

Per the instructor's email announcement above, this question is also waived and no response is submitted for this section.

6. AI Usage Declaration

TODO: Declare any AI tool use as per the course policy. If none, write: "I did not use any AI tools in completing this sprint."

- **Tool:** *e.g., ChatGPT-4o*
What I asked: ...
What I understood and applied: ...